Faculdade de Ciências da Universidade do Porto

Desenvolvimento de um Sistema Automático para Deteção de Ficheiros Duplicados e Inconsistentes

ARMIS Group

ZhiXu Ni



Licenciatura em Inteligência Artificial e Ciência de Dados

Orientador: Joel Carneiro

Orientador FCUP: Prof. Eduardo Resende Brandão Marques

29 de junho de 2026

Resumo

A receção de grandes volumes de ficheiros provenientes de múltiplos clientes levanta desafios significativos ao nível da identificação de duplicados, ficheiros semelhantes e inconsistências estruturais.

Este trabalho tem como objetivo desenvolver um sistema automático para deteção de ficheiros duplicados e inconsistentes, capaz de operar de forma eficiente em diferentes tipos de dados, incluindo texto, imagens, áudio e vídeo.

A abordagem adotada baseia-se na combinação de múltiplas técnicas de hashing e análise de similaridade, nomeadamente hashing criptográfico (SHA-256 e BLAKE3) para deteção de duplicados exatos, fuzzy hashing para identificação de semelhanças em ficheiros binários, hashing perceptual para conteúdos multimédia e algoritmos de similaridade textual, como SimHash e MinHash, aplicados a documentos estruturados.

O sistema desenvolvido integra um pipeline composto pelas etapas de validação, indexação e deteção, permitindo identificar automaticamente duplicados, ficheiros semelhantes e inconsistências, como formatos inválidos ou dados corrompidos.

Os resultados demonstram que a utilização combinada destas técnicas permite alcançar um bom equilíbrio entre desempenho e precisão, tornando o sistema adequado para cenários reais de processamento automatizado de ficheiros.

Conclui-se que a solução proposta contribui significativamente para a automatização da gestão de dados, reduzindo o esforço manual e melhorando a eficiência operacional em ambientes empresariais.

Agradecimentos

Gostaria de expressar o meu agradecimento a todas as pessoas e entidades que contribuíram para a realização deste trabalho.

Em primeiro lugar, ao meu orientador, Joel Carneiro, pelo acompanhamento contínuo, apoio técnico e orientação ao longo de todo o desenvolvimento do projeto.

Ao Professor Eduardo Resende Brandão Marques, pelo apoio académico e contributo na validação do trabalho desenvolvido.

À empresa ARMIS Group, pela oportunidade concedida, pelo contexto real de aplicação e pelos recursos disponibilizados, que permitiram o desenvolvimento deste sistema em ambiente prático.

Por fim, um agradecimento a todos os colegas e familiares pelo apoio e motivação ao longo deste percurso.

Índice

1 Introdução **1**

2 Estado da Arte **2**

2.1 Tipos de Ficheiros Envolvidos 2

2.2 Técnicas de Deteção 2

2.2.1 Deteção de Duplicados 2

2.2.2 Deteção de Ficheiros Semelhantes 3

2.3 Deteção de Inconsistências 4

3 Metodologia **6**

3.1 Abordagem de Desenvolvimento 6

3.2 Atividades desenvolvidas 6

3.3 Dificuldades Encontradas 6

3.4 Intervenientes 6

4 Solução **7**

4.1 Requisitos do Sistema 7

4.2 Arquitetura Geral 7

4.2.1 Camada de Utilitários 7

4.2.2 Fase de Validação 7

4.2.3 Fase de Indexação 8

4.2.4 Fase de Deteção 9

4.3 Execução do Sistema 9

5 Resultados **11**

5.1 Dataset e Metodologia de Avaliação 11

5.1.1 Desempenho por Método 11

5.1.2 Desempenho Global 12

5.1.3 Avaliação dos Métodos 12

5.2 Exemplo de Execução no Modo Interativo 14

6 Conclusões **15**

6.1 Resultados alcançados 15

6.2 Lições aprendidas 15

6.3 Trabalho futuro 15

Referências **16**

Lista de Figuras

2.1 Processo de hashing: a entrada é convertida numa assinatura de tamanho fixo (Codecademy 2025). 3

2.2 Fuzzy hashing: ficheiros semelhantes originam assinaturas próximas, permitindo quantificar a similaridade. Fonte adaptada de (Tatum et al. 2024). 3

2.3 Hashing perceptual: aHash (esquerda) baseado na intensidade média; dHash (direita) baseado em diferenças entre pixels adjacentes. Adaptado de `https://medium.com/data-science-collective/detection-of-near-identical-images-using-image-hash-functions-c61e133a0958`. 4

2.4 SequenceMatcher: a similaridade é calculada como 2×match size/total size. Adaptado de `https://www.maxlist.xyz/2022/11/05/sequencematcher/`. 4

5.1 Comparação do tempo de execução por método, evidenciando a contribuição das fases de indexação, deteção e tempo total. 11

5.2 Comparação global entre os pipelines `run_all` e `run_optimized`, destacando as fases de validação, indexação e deteção. 12

5.3 Comparação da taxa de sucesso na deteção de duplicados e ficheiros semelhantes por método. 13

Lista de Tabelas

4.1 Validadores implementados por categoria de ficheiro. 8

5.1 Tempo de execução dos métodos, discriminado pelas fases de indexação e deteção. 11
5.2 Tempo total de execução dos pipelines e respetiva distribuição por fases. 12
5.3 Eficácia de deteção por método e desempenho global do pipeline otimizado. . . . 13
5.4 Exemplo de execução no modo interativo. 14

Glossário

Hashing criptográfico
Função que gera uma assinatura fixa (ex.: SHA-256) para identificar duplicados exatos.

Fuzzy hashing
Técnica que estima similaridade entre ficheiros com pequenas alterações.

Hash perceptual (pHash)
Método que extrai características de conteúdos multimédia de forma robusta.

SimHash
Algoritmo que representa texto para comparação via distância de Hamming.

TLSH / SSDEEP
Algoritmos de fuzzy hashing para medir similaridade em dados binários.

Fingerprint
Representação compacta de um ficheiro usada para comparação.

Duplicado exato
Ficheiro com conteúdo binário idêntico a outro.

Ficheiro semelhante
Ficheiro com diferenças, mas conteúdo parcialmente comum.

Pipeline
Sequência de processamento: validação, indexação e deteção.

FAST → DEEP
Estratégia que aplica métodos rápidos antes dos mais complexos.

Indexação
Criação de assinaturas para todos os ficheiros.

Deteção
Comparação de ficheiros para identificar duplicados ou similaridade.

Validação
Verificação da integridade e formato dos ficheiros.

Score de similaridade
Valor numérico que indica o grau de semelhança entre ficheiros.

Threshold
Valor limite para classificar ficheiros como semelhantes.

MIME type
Tipo de ficheiro identificado pelo conteúdo (ex.: `image/jpeg`).

Index global
Base de fingerprints reutilizada em execuções futuras.

Modo interativo
Processamento incremental de ficheiros em tempo real.

Introdução

Contexto

O presente trabalho foi desenvolvido no âmbito de um estágio na empresa ARMIS Group, num contexto de processamento de grandes volumes de ficheiros provenientes de múltiplos clientes, incluindo documentos, imagens, áudio e vídeo.

A crescente quantidade de dados cria necessidades de organização e controlo de qualidade, tornando a deteção manual de duplicados ou inconsistências impraticável.

Problema

Identificam-se três problemas principais:

(1) ficheiros duplicados com nomes distintos;

(2) ficheiros semelhantes, onde pequenas alterações impedem a deteção binária;

(3) ficheiros inconsistentes, com dados corrompidos ou formatos inválidos.

Nenhuma técnica isolada cobre estes cenários de forma eficiente, sendo necessária uma abordagem combinada.

Objetivos

O objetivo é desenvolver um sistema automático capaz de:

- detetar duplicados e ficheiros semelhantes em múltiplos formatos;
- validar a integridade e consistência dos ficheiros;
- operar de forma eficiente, minimizando o custo computacional.

A solução baseia-se na combinação de hashing criptográfico, fuzzy hashing e hashing perceptual, integrados num pipeline modular.

Acesso ao Código

O código desenvolvido encontra-se disponível em:

`https://github.com/zni0218/Ficheiros-Duplicados`

Estrutura do Relatório

O Capítulo 2 apresenta o estado da arte. O Capítulo 3 descreve a metodologia. O Capítulo 4 detalha a solução. O Capítulo 5 apresenta os resultados. O Capítulo 6 expõe as conclusões.

Estado da Arte

Este capítulo apresenta o enquadramento técnico das principais abordagens utilizadas na deteção de ficheiros duplicados, semelhantes e inconsistentes.

2.1 Tipos de Ficheiros Envolvidos

Em contextos empresariais semelhantes, os sistemas recebem ficheiros de múltiplas categorias: documentos (XML, JSON, CSV, PDF, DOCX), imagens (JPG, PNG, TIFF), ficheiros de áudio (MP3, WAV, FLAC), vídeo (MP4, MOV, AVI), logs (TXT) e arquivos compactados (ZIP, RAR).

O sistema deve ser suficientemente genérico para lidar com esta diversidade.

2.2 Técnicas de Deteção

As técnicas de deteção variam consoante o tipo de ficheiro e o nível de semelhança pretendido, baseando-se principalmente em métodos de hashing.

Algoritmos de Hashing

Os algoritmos de hashing criptográfico transformam uma entrada x num valor de tamanho fixo: $h = H(x)$, As principais propriedades incluem:

- **Resistência à colisão:** difícil encontrar $x_1 \neq x_2$ tal que $H(x_1) = H(x_2)$;
- **Resistência à pré-imagem:** dado h, difícil encontrar x tal que $H(x) = h$;
- **Resistência à segunda pré-imagem:** dado x_1, difícil encontrar $x_2 \neq x_1$ tal que $H(x_1) = H(x_2)$.

2.2.1 Deteção de Duplicados

Hashing Criptográfico Tradicional

Utilizado para identificar ficheiros binariamente idênticos.

Os algoritmos mais utilizados são MD5 (Rivest 1992), SHA-256 (National Institute of Standards and Technology 2015) e BLAKE3 (BLAKE3 Team 2024)

Como ilustrado na Figura 2.1, o hashing transforma dados de dimensão arbitrária numa assinatura de tamanho fixo.

Pequenas alterações no ficheiro resultam em hashes completamente distintos (efeito avalanche).

Hashing Parcial

Consiste no cálculo do hash sobre partes do ficheiro os primeiros e últimos blocos / blocos selecionados. É mais rápido, mas menos fiável.

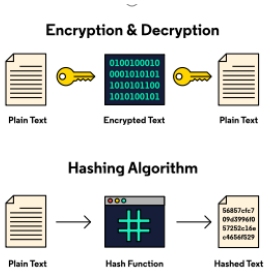


Figura 2.1: Processo de hashing: a entrada é convertida numa assinatura de tamanho fixo (Codecademy 2025).

Metadados

Incluem tamanho, extensão, tipo MIME, datas e nome. São rápidos, mas pouco robustos isoladamente.

2.2.2 Deteção de Ficheiros Semelhantes

Em cenários reais, ficheiros podem diferir por compressão ou formatação.

Fuzzy Hashing

O fuzzy hashing permite identificar ficheiros semelhantes através de assinaturas aproximadas, tolerantes a pequenas alterações no conteúdo (Microsoft Security 2021; Di et al. 2016).

Como ilustrado na Figura 2.2, ficheiros com ligeiras diferenças produzem assinaturas próximas, mensuráveis por uma distância de similaridade.

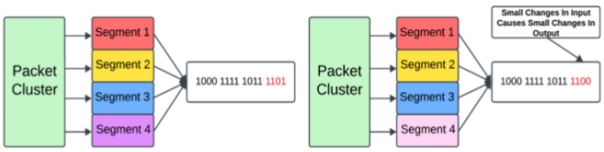


Figura 2.2: Fuzzy hashing: ficheiros semelhantes originam assinaturas próximas, permitindo quantificar a similaridade. Fonte adaptada de (Tatum et al. 2024).

Os principais algoritmos são: ssdeep (OI 2023; James 2017), que divide o ficheiro em blocos e calcula a similaridade por comparação de segmentos; TLSH (Oliver 2021; Oliver e Hagen 2021), que analisa a distribuição estatística dos bytes; e SDHash (Roussev 2023), baseado em características raras do conteúdo.

Hashing Perceptual

Aplicado a conteúdos multimédia, o hashing perceptual captura características visuais ou acústicas relevantes, sendo robusto a transformações como redimensionamento ou recompressão.

Os algoritmos aHash, pHash e dHash (Zauner 2010; Jie 2013; Buchner 2023; Jain et al. 2019) são os mais utilizados para imagens.

Como ilustrado na Figura 2.3, cada algoritmo representa a imagem de forma binária com base em características distintas: o aHash usa a intensidade média dos pixels, enquanto o dHash compara pixels adjacentes.

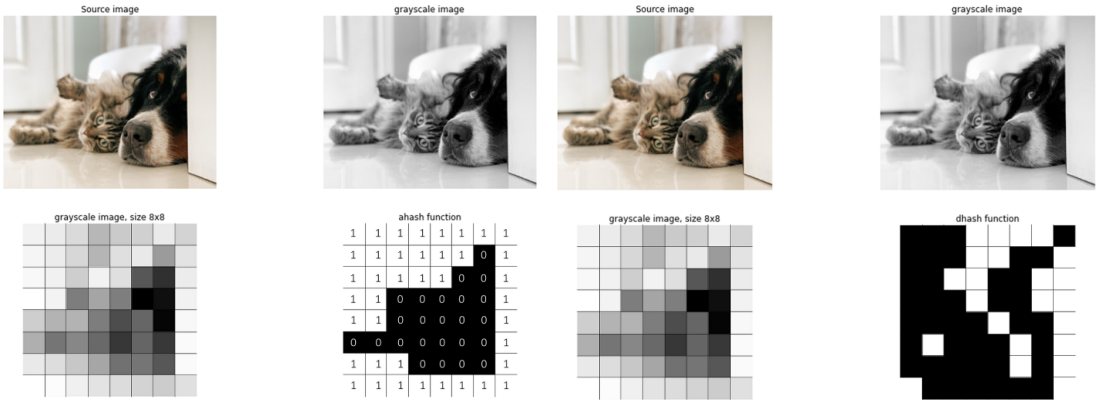


Figura 2.3: Hashing perceptual: aHash (esquerda) baseado na intensidade média; dHash (direita) baseado em diferenças entre pixels adjacentes. Adaptado de `https://medium.com/data-science-collective/detection-of-near-identical-images-using-image-hash-functions-c61e133a0958`.

Aplicação a Vídeo e Áudio

- Vídeo: frames representativos;
- Áudio: espectrogramas.

Após este pré-processamento, é possível aplicar hashing perceptual.

Similaridade Textual

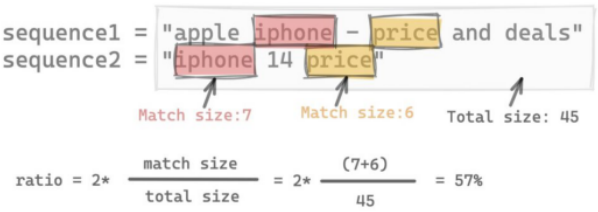


Figura 2.4: SequenceMatcher: a similaridade é calculada como 2 × match size/total size. Adaptado de `https://www.maxlist.xyz/2022/11/05/sequencematcher/`.

Para documentos textuais, técnicas como TF-IDF, similaridade de Jaccard e distância de Levenshtein (Manning e Raghavan 2008; Scikit-learn 2023) permitem quantificar diferenças.

SimHash e MinHash (Charikar 2002; Shrivastava 2014) são especialmente adequados para documentos de grande dimensão, gerando representações compactas que suportam comparação eficiente pela distância de Hamming.

O algoritmo SequenceMatcher (Docs 2023), ilustrado na Figura 2.4, calcula a similaridade com base em segmentos correspondentes entre duas sequências.

2.3 Deteção de Inconsistências

Identifica ficheiros inválidos ou inconsistentes.

Validação Estrutural

- assinaturas e cabeçalhos (magic numbers) e leitura através de parsers específicos.

Tamanhos Anómalos

- deteção de outliers e limites por tipo.

Formatos Inesperados

Verifica correspondência entre extensão e tipo MIME:

- ferramenta `file` e biblioteca `python-magic` (Hupp 2023).

Metodologia

3.1 Abordagem de Desenvolvimento

O desenvolvimento seguiu uma metodologia iterativa e incremental, permitindo implementar funcionalidades de forma progressiva com validação contínua dos resultados.

As iterações decorreram em ciclos de análise, implementação e teste, com reuniões regulares com os orientadores para revisão do progresso.

A implementação foi realizada em Python, com controlo de versões em Git, assegurando rastreabilidade das alterações e facilidade de reversão em caso de erros.

3.2 Atividades desenvolvidas

As principais atividades foram:

- Análise do problema e requisitos;
- Estudo de técnicas de deteção;
- Implementação de métodos de hashing;
- Desenvolvimento do pipeline;
- Testes e avaliação do sistema.

3.3 Dificuldades Encontradas

Durante o desenvolvimento, surgiram obstáculos técnicos relevantes.

A biblioteca `ssdeep` apresentou problemas de instalação que exigiram configuração adicional do ambiente.

A utilização de `python-magic` originou conflitos entre versões da biblioteca, afetando a deteção de tipos MIME.

Estas situações foram resolvidas através da documentação das dependências e da definição de mecanismos de fallback, como a deteção de tipo por extensão quando o MIME não estava disponível.

3.4 Intervenientes

O projeto foi desenvolvido no âmbito de um estágio na ARMIS Group.

O autor foi responsável pela implementação do sistema e realização dos testes.

O trabalho foi acompanhado por orientadores empresarial e académico.

Solução

Neste capítulo é apresentada a solução desenvolvida para a deteção automática de ficheiros duplicados, semelhantes e inconsistentes, baseada numa arquitetura modular implementada em Python.

4.1 Requisitos do Sistema

Os requisitos funcionais principais são a deteção de duplicados exatos e semelhantes, e a validação de integridade de ficheiros, com suporte a múltiplos formatos.

Os requisitos não funcionais incluem eficiência de execução, escalabilidade, modularidade e robustez.

As principais restrições são a dependência de bibliotecas externas (ex.: TLSH, ssdeep) e a aplicabilidade de certos métodos apenas a categorias específicas de ficheiros.

4.2 Arquitetura Geral

O sistema segue um pipeline composto por três fases principais:

- **Validação**: análise da integridade e consistência de formato dos ficheiros;
- **Indexação**: geração de assinaturas (*fingerprints*) através de diferentes métodos;
- **Deteção**: comparação entre fingerprints e classificação de similaridade.

A arquitetura é modular, permitindo a integração e substituição de métodos de forma independente.

4.2.1 Camada de Utilitários

O sistema inclui uma camada transversal de suporte que assegura consistência e robustez.

O módulo `path_utils.py` garante normalização de caminhos com compatibilidade entre Windows e Linux.

O módulo `file_utils.py` implementa a deteção de tipo MIME, o fallback por extensão, e o cálculo de hashing por blocos.

O módulo `score_utils.py` trata da normalização de scores, definição de thresholds e classificação final dos pares comparados.

4.2.2 Fase de Validação

O módulo `run_validation.py` coordena a validação de todos os ficheiros, recorrendo ao módulo `general_validation.py` para verificações específicas por tipo.

O processo inclui: deteção do tipo MIME a partir do conteúdo; verificação de consistência entre extensão e tipo real; e validação estruturada por categoria (Tabela 4.1).

Categoria	Biblioteca / Método
Documentos PDF	`PyPDF2`
Ficheiros comprimidos	`ZipFile`
Imagens	`PIL`
Áudio	`soundfile`
Vídeo	`OpenCV`
JSON	parsing completo

Tabela 4.1: Validadores implementados por categoria de ficheiro.

Cada ficheiro é classificado como *válido* ou *inválido*, com registo do motivo da falha e do tempo de validação. Os resultados são exportados em ficheiros CSV organizados por categoria.

4.2.3 Fase de Indexação

A indexação é realizada através do módulo `build_index.py`, responsável pela geração de assinaturas (*fingerprints*) para cada ficheiro.

O processo inclui:

- filtragem com base no tipo de ficheiro (MIME);
- aplicação de múltiplos métodos de hashing;
- registo do tempo de execução;
- armazenamento estruturado dos resultados.

As assinaturas são agregadas no ficheiro `combined_index.csv`, permitindo uma visão unificada e suportando atualização incremental do índice.

Foram implementadas diferentes técnicas de hashing, adaptadas ao tipo de ficheiro e ao nível de similaridade pretendido:

- **Hashing criptográfico** (`hashing_exato`): SHA-256 para duplicados exatos;
- **Fuzzy hashing** (`ssdeep`, `tlsh`): similaridade entre ficheiros binários;
- **Hashing perceptual** (`image_phash`, `audio_phash`, `video_phash`): para imagens, áudio (via espectrograma) e vídeo (frames representativos);
- **Similaridade textual** (`text_simhash`): SimHash com distância de Hamming;
- **Hashing por blocos** (`fuzzy_chunks`): divisão em segmentos com amostragem.

Importa salientar que nem todos os métodos podem ser aplicados a todos os tipos de ficheiros. Cada técnica é projetada para operar sobre categorias específicas, de acordo com o formato dos dados. Por exemplo, métodos de hashing perceptual como `image_phash`, `audio_phash` e `video_phash` são exclusivamente aplicados a ficheiros multimédia, enquanto `text_simhash` é limitado a dados textuais. De forma semelhante, métodos como `ssdeep` e `tlsh` operam sobre dados binários.

4.2.4 Fase de Deteção

O módulo `run_detection.py` compara pares de ficheiros usando os fingerprints gerados.

A comparação é restringida à mesma categoria de ficheiro, reduzindo o número de combinações.

As métricas de similaridade variam por método: comparação direta de hashes (SHA-256), distância de Hamming (pHash, SimHash), score de similaridade (ssdeep), distância TLSH, e razão de correspondência (SequenceMatcher).

Os scores são normalizados para $[0, 1]$, e cada par é classificado como: *duplicado exato* (hashes SHA-256 idênticos), *semelhante forte*, *semelhante* (acima do threshold mínimo), ou *baixa similaridade*.

Resultados

Cada comparação produz um registo contendo:

- ficheiros comparados;
- método utilizado;
- score bruto e normalizado;
- classificação;
- tempo de execução.

Esta estrutura permite analisar o desempenho dos diferentes métodos de forma consistente.

4.3 Execução do Sistema

A solução foi desenvolvida para suportar diferentes modos de execução, permitindo tanto análise completa de datasets como simulação de cenários reais de utilização.

Execução Completa (run_all)

O módulo `run_all.py` implementa o pipeline completo de processamento:

validação → indexação → deteção

Este modo é utilizado para análise global de datasets, executando todos os métodos disponíveis.

O processo inclui:

- validação de todos os ficheiros;
- construção completa do índice com múltiplos métodos;
- comparação entre todos os ficheiros;
- geração de resultados por método;
- cálculo de métricas de desempenho.

Os resultados são armazenados em diferentes ficheiros CSV:

- resultados por método;
- resultados agregados (duplicados, semelhantes);
- métricas de desempenho por método;
- métricas globais do pipeline.

Execução Otimizada (`run_optimized`)

O módulo `run_optimized.py` implementa uma versão otimizada do pipeline completo, adotando uma estratégia de processamento (FAST → DEEP).

Para otimizar o desempenho, a deteção é dividida em duas fases:

- **FAST**: utiliza hashing criptográfico (`SHA-256`) para detetar duplicados exatos de forma imediata;
- **DEEP**: aplica métodos mais avançados apenas quando não é encontrado duplicado.

Métodos utilizados no DEEP:

- `tlsh`, `image_phash`, `text_simhash`, `audio_phash`, `video_phash`

Esta abordagem permite evitar a aplicação de métodos mais dispendiosos em ficheiros já identificados como duplicados, reduzindo significativamente o custo computacional sem comprometer a precisão.

Modo Interativo

O módulo `interactive_compare.py` simula um ambiente de produção, onde os ficheiros são recebidos continuamente.

Este modo utiliza uma estratégia otimizada semelhante ao pipeline `run_optimized`, aplicando inicialmente métodos rápidos e recorrendo a métodos mais complexos apenas quando necessário.

O processamento segue um fluxo incremental:

$$\text{INPUT} \rightarrow \text{VALIDATION} \rightarrow \text{FAST} \rightarrow (\text{STOP ou DEEP}) \rightarrow \text{RESULTADO} \rightarrow \text{UPDATE}$$

O sistema mantém um índice incremental da sessão e suporta opcionalmente integração com um índice global pré-construído. Para cada ficheiro processado, são apresentados em tempo real o tipo de correspondência (duplicado ou semelhante), a origem (sessão ou global) e o tempo de processamento.

Resultados

A avaliação do sistema foi realizada através de testes experimentais com base nos resultados obtidos a partir do pipeline completo (`run_all`) e da sua versão otimizada (`run_optimized`).

5.1 Dataset e Metodologia de Avaliação

O sistema foi avaliado sobre um dataset de 83 ficheiros de diferentes categorias (áudio, imagem, texto, vídeo, binário e documentos).

Os resultados par-a-par, armazenados em `ALL_combined.csv`, foram agregados ao nível de ficheiro, identificando 42 ficheiros duplicados, 59 ficheiros semelhantes e 10 ficheiros sem qualquer relação.

Para garantir consistência, foi aplicada uma abordagem de *cross-validation* sobre esta representação.

5.1.1 Desempenho por Método

Método	Index (s)	Detection (s)	Total (s)
hashing_exato	0.8463	0.0002	0.8465
image_phash	0.2445	0.0104	0.2549
audio_phash	11.1973	0.0016	11.1989
text_simhash	0.0300	0.0020	0.0320
video_phash	3.4885	0.0266	3.5151
ssdeep	29.7047	0.0025	29.7072
tlsh	5.2625	0.0005	5.2630
fuzzy_chunks	0.1166	0.0241	0.1407

Tabela 5.1: Tempo de execução dos métodos, discriminado pelas fases de indexação e deteção.

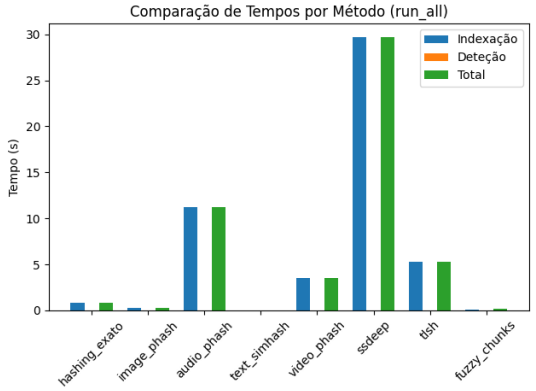


Figura 5.1: Comparação do tempo de execução por método, evidenciando a contribuição das fases de indexação, deteção e tempo total.

Na Tabela 5.1 e na Figura 5.1 observa-se que a fase de indexação representa a componente dominante do tempo de execução em todos os métodos, enquanto a fase de deteção apresenta um custo residual.

Verifica-se ainda uma variação significativa no custo computacional entre métodos. Técnicas aplicadas a ficheiros multimédia, como `audio_phash` e `video_phash`, apresentam tempos elevados devido ao processamento necessário para extração de características. O método `ssdeep` destaca-se como o mais dispendioso.

Por outro lado, métodos como `text_simhash` e `image_phash` apresentam tempos reduzidos, sendo mais eficientes. No entanto, estes resultados dependem das características do dataset, nomeadamente da quantidade e dimensão dos ficheiros analisados em cada categoria.

5.1.2 Desempenho Global

Pipeline	Total (s)	Validation (s)	Index (s)	Detection (s)
run_all	52.8989	0.7272	52.0151	0.1093
run_optimized	22.7607	0.6199	22.0463	0.0569

Tabela 5.2: Tempo total de execução dos pipelines e respetiva distribuição por fases.

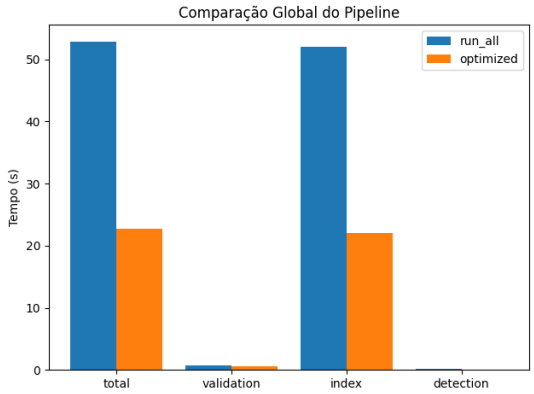


Figura 5.2: Comparação global entre os pipelines `run_all` e `run_optimized`, destacando as fases de validação, indexação e deteção.

Os resultados apresentados na Tabela 5.2 e na Figura 5.2 mostram que a fase de indexação domina o custo computacional em ambos os pipelines.

O pipeline otimizado reduz o tempo total de execução de 52.90s para 22.76s, correspondendo a uma melhoria superior a 50%.

Esta redução resulta da estratégia FAST → DEEP, que evita a aplicação de métodos mais dispendiosos em casos já resolvidos. Em particular, ficheiros identificados como duplicados na fase inicial não são submetidos a etapas mais complexas, reduzindo significativamente o número de operações realizadas.

5.1.3 Avaliação dos Métodos

A Tabela 5.3 e a Figura 5.3 apresentam a eficácia dos métodos na deteção de duplicados e ficheiros semelhantes.

Método	Duplicados (%)	Semelhantes (%)	Tempo (s)
hashing_exato	71.43	0.00	0.8465
image_phash	100.00	100.00	0.2549
audio_phash	100.00	100.00	11.1989
text_simhash	100.00	100.00	0.0320
video_phash	100.00	100.00	3.5151
ssdeep	71.43	22.58	29.7072
tlsh	71.79	55.56	5.2630
fuzzy_chunks	71.43	0.00	0.1407
pipeline_run_all	–	–	52.8989
pipeline_optimized	100.00	100.00	22.7607

Tabela 5.3: Eficácia de deteção por método e desempenho global do pipeline otimizado.

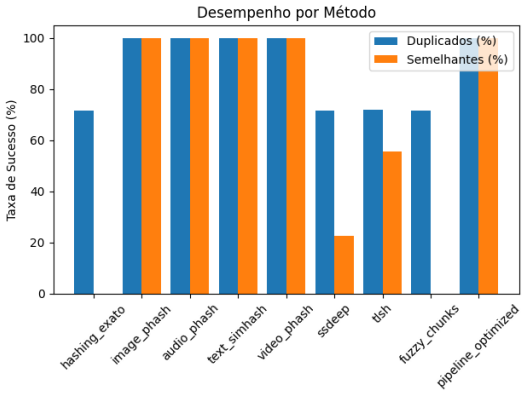


Figura 5.3: Comparação da taxa de sucesso na deteção de duplicados e ficheiros semelhantes por método.

A análise evidencia diferenças claras entre abordagens. O `hashing_exato` apresenta bom desempenho na deteção de duplicados, mas não identifica ficheiros semelhantes. Os métodos de fuzzy hashing, como `ssdeep` e `tlsh`, revelam desempenho intermédio na deteção de similaridade.

Os métodos perceptuais e textuais atingem taxas de sucesso elevadas nas respetivas categorias, mas não são diretamente comparáveis entre si, uma vez que operam sobre tipos de dados distintos.

O pipeline otimizado destaca-se por integrar automaticamente diferentes métodos, permitindo analisar o dataset de forma consistente. Como resultado, atinge uma taxa de deteção de 100% com um tempo inferior ao pipeline completo.

Estes resultados confirmam a eficácia da estratégia FAST → DEEP, permitindo reduzir o custo computacional sem comprometer a precisão.

5.2 Exemplo de Execução no Modo Interativo

Para demonstrar o funcionamento do sistema em contexto real, apresenta-se um exemplo de execução do módulo `interactive_compare.py`, no qual os ficheiros são processados sequencialmente.

Neste modo, cada novo ficheiro é comparado com os já existentes na sessão, seguindo uma abordagem incremental baseada na estratégia FAST → DEEP.

Ficheiro	Resultado	Tempo (ms)
video_original.mp4	Sem correspondência	128.71
video_original_copy.mp4	Duplicado	22.15
video_2.mp4	Sem correspondência	114.92
video_144p.mp4	Similar	299.39
video_720p.mp4	Similar forte	1043.53

Tabela 5.4: Exemplo de execução no modo interativo.

Como se observa na Tabela 5.4, o sistema é capaz de identificar diferentes tipos de correspondência, nomeadamente duplicados exatos, ficheiros semelhantes e casos sem correspondência.

Adicionalmente, verifica-se que o tempo de processamento varia consoante o tipo de resultado obtido. Ficheiros identificados como duplicados exatos apresentam tempos significativamente mais reduzidos, uma vez que são detetados na fase FAST.

Por outro lado, a deteção de ficheiros semelhantes requer a aplicação de métodos mais complexos na fase DEEP, resultando em tempos mais elevados.

Este comportamento confirma a eficácia da estratégia FAST → DEEP, permitindo obter respostas rápidas sempre que possível e reservando processamento adicional apenas para os casos em que é necessário analisar similaridade.

Conclusões

6.1 Resultados alcançados

O trabalho desenvolvido permitiu implementar um sistema completo de deteção de ficheiros duplicados e semelhantes, suportando múltiplos tipos de dados.

A análise evidenciou que nenhum método isolado garante cobertura total: o hashing criptográfico falha na deteção de ficheiros semelhantes, o fuzzy hashing apresenta cobertura limitada, e os métodos perceptuais e textuais estão restritos às respetivas categorias. Assim, a combinação estratégica destes métodos, com priorização baseada no custo computacional, revelou-se a abordagem mais eficaz.

Neste contexto, a principal contribuição do trabalho consiste na definição de um pipeline otimizado baseado na estratégia $FAST \rightarrow DEEP$, que permite reduzir significativamente o custo computacional ao evitar processamento desnecessário.

Os resultados demonstraram que o pipeline otimizado reduz o tempo total de execução de aproximadamente 52.9 segundos para 22.8 segundos, mantendo simultaneamente uma taxa de deteção de 100% para ficheiros duplicados e semelhantes.

6.2 Lições aprendidas

Durante o desenvolvimento, verificou-se a importância de combinar diferentes métodos de análise para lidar com a diversidade dos dados.

Observou-se também que o desempenho dos algoritmos depende fortemente das características do dataset, sendo necessária uma análise crítica dos resultados para evitar conclusões enviesadas.

A implementação do pipeline otimizado demonstrou a relevância de estratégias que reduzam a complexidade computacional, nomeadamente através da eliminação de cálculos redundantes e da priorização de métodos mais eficientes.

6.3 Trabalho futuro

Como trabalho futuro, propõe-se:

- melhorar a generalização dos métodos para diferentes tipos de ficheiros e datasets mais heterogéneos;
- explorar abordagens baseadas em aprendizagem automática para deteção de similaridade;
- adaptar o sistema a cenários distribuídos e de maior escala;
- desenvolver uma API e uma aplicação que permita integrar o sistema em ambientes reais, facilitando a sua utilização direta em pipelines de processamento e sistemas empresariais.

Referências bibliográficas

BLAKE3 Team (2024). *BLAKE3 Cryptographic Hash Function*. URL: https://github.com/BLAKE3-team/BLAKE3.

Buchner, Johannes (2023). *ImageHash Python Library*. URL: https://github.com/JohannesBuchner/imagehash.

Charikar, Moses (2002). *Detecting Near-Duplicates for Web Crawling*. URL: https://dl.acm.org/doi/10.1145/509907.509965.

Codecademy (2025). *What is Hashing*. URL: https://www.codecademy.com/resources/blog/what-is-hashing.

Di, Hongyu et al. (2016). «Research on Document Comparison Algorithm Based on Modified Fuzzy Hash». Em: *Journal of Network Information Security*. URL: https://doi.org/10.3969/j.issn.1671-1122.2016.11.003.

Docs, Python (2023). *SequenceMatcher Explanation*. URL: https://docs.python.org/3/library/difflib.html.

Hupp, A. (2023). *python-magic*. URL: https://github.com/ahupp/python-magic.

Jain, Tanuj et al. (2019). *Imagededup*. URL: https://github.com/idealo/imagededup.

James, Joshua I. (2017). *How To: Fuzzy Hashing with SSDEEP (Similarity Matching)*. URL: https://dfir.science/2017/07/How-To-Fuzzy-Hashing-with-SSDEEP-(similarity-matching).html.

Jie, Zeng (2013). *Perceptual Hashing for Image Similarity*. URL: https://doi.org/10.48550/arXiv.1306.4079.

Manning, Christopher e Prabhakar Raghavan (2008). *Introduction to Information Retrieval*. URL: https://nlp.stanford.edu/IR-book/.

Microsoft Security (2021). *Using Fuzzy Hashing for Malware Detection*. URL: https://www.microsoft.com/en-us/security/blog/2021/07/27/combing-through-the-fuzz-using-fuzzy-hashing.

National Institute of Standards and Technology (2015). *Secure Hash Standard (SHS)*. URL: https://csrc.nist.gov/publications/detail/fips/180/4/final.

OI, Tsukasa (2023). *ssdeep - Fuzzy Hashing Program*. URL: https://ssdeep-project.github.io/ssdeep/index.html.

Oliver, Jonathan (2021). *TLSH Papers and Research*. URL: https://tlsh.org/papers.html.

Oliver, Jonathan e Josiah Hagen (2021). *Designing the Elements of a Fuzzy Hashing Scheme*. URL: https://doi.org/10.1109/EUC53437.2021.00028.

Rivest, Ronald L. (1992). *The MD5 Message-Digest Algorithm*. URL: https://www.rfc-editor.org/rfc/rfc1321.

Roussev, Vassil (2023). *SDHash Project*. URL: http://roussev.net/sdhash/sdhash.html.

Scikit-learn (2023). *Jaccard Similarity*. URL: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.jaccard_score.html.

Shrivastava, Anshumali (2014). *Improved Consistent Sampling, Weighted Minhash*. URL: https://doi.org/10.48550/arXiv.1407.4416.

Tatum, William et al. (2024). *Anomaly Detection in ICS Networks with Fuzzy Hashing*. URL: https://doi.org/10.1109/CARS61786.2024.10778919.

Zauner, Christoph (2010). *Implementation and Benchmarking of Perceptual Image Hash Functions*. URL: https://api.semanticscholar.org/CorpusID:17075066.